

Assignment 3

With the project shipped, the client wants your team to take over one of their other projects that they gained after an acquisition.

Both teams from this lab will send all their work to the subgroup from the list below (more will be added for each lab). You will also receive 2 solutions once the appropriate subgroup had their lab. You will decide which team will work on which problem. This change is permanent; all future work will be done on this codebase. The swap order was chosen with the intention to make the final merge easier. For this to happen the project is moved from a group to teams that have their labs next to each other. The format is Who created the project => Who they need to send it to.

925/1 => 922/1	927/1 => 922/2	922/1 => 925/1	924/1 => 923/2
922/2 => 927/1	923/1 => 927/2	926/1 => 921/2	925/2 => 924/1
927/2 => 923/1	MIE+E <=>MIE3+E	921/1 => 926/1	921/2 => 925/2
924/2 => 926/2	923/2 => 921/1	926/2 => 924/2	

- Make any architectural changes you deem relevant. Fix what is not working, it will make later assignments more manageable. Feel free to use design patterns if you want.
- Refactor the code to make it more readable: all functions and variables must have a semantic meaning whenever possible, there may be no abbreviations (not even in lambdas). Deal with magic numbers.
- Make sure there is no business logic in GUI. If you find some then move it to a non-GUI class.
- No layer shall know directly about the next one, use interfaces to separate them.
- Write unit tests in the manner presented at the course. You should have 100% coverage on the parts that you should, and none elsewhere. Add integration tests wherever appropriate. You may use isolation frameworks. Your automated test must have the same quality and uniformity as the rest of the codebase.
- Use styleCop static code analysis to find issues in the code and fix them, this will also help to homogenize the various projects a bit so merging them will be less painful.
 1. Copy the SE.ruleset file from teams to your project(s) root.
 2. Right click on project(s)-> manage NugetPackages
 3. On the Browse tab search for -> StyleCop.Analyzers, then install
 4. Right click on your project -> Edit project file
 5. Search for the first <PropertyGroup> and add the line:
`<CodeAnalysisRuleSet>SE.ruleset</CodeAnalysisRuleSet>`
 6. Build/attempt to run
- If you make a new GitHub repo there is no need to share it with your lab assistant.

Ongoing requirements:

- The team lead will present a report before demoing their work on the contributions of each team member, ideally task/git/chat/sent files history etc. With the scope of proving that each team member did some non-negligible amount of work.
- SQL queries must still be absent of business logic.

Optional:

- Try to start early and target getting everything done a couple of days before the deadline.
- Use tasks for work tracking and management.

Deadline: lab 4